A Public End-to-End Full-Key Recovery Attack on the `main.c` Implementation of SEPAR

2026-04-14

Abstract

We study the *implementation* of SEPAR in `SEPAR/main.c`, not the abstract cipher description. In the chosen-IV, chosen-plaintext, chosen-ciphertext, reset model, we give a public end-to-end full-key recovery attack against the 256-bit implementation key. The attack is exact once a suitable IV is found. Its only stochastic component is the chosen-IV search. The deterministic recovery pipeline has three parts: an exact stage-8 public bootstrap; an exact same-stage inward recursion that recovers $(K_7, s_8), \ldots, (K_2, s_3)$; and an exact stage-1 closure from the recovered stage-2 reduced table. The final acceptance condition is exact: a candidate 256-bit key is accepted if and only if it reproduces the full one-word oracle table on the successful IV. We give exact mathematical definitions for every primitive used by the implementation, state the full attack as a sequence of explicit public transformations on 2^{16}-entry tables, prove conditional correctness, report a complete random-key recovery, and give a Monte Carlo estimate for the success probability of the IV-search phase.

1 Introduction

The target of this paper is the concrete word-oriented implementation in `SEPAR/main.c`. The relevant attack surface is not the abstract design claim; it is the resettable 16-bit one-word interface induced by the code. Once an IV is fixed, the implementation exposes an exact public permutation of $\mathbb{Z}_{2^{16}}$ on one-word inputs. Because the block size is only 16 bits, one can recover and manipulate the full codebook exactly.

The attack model is:

- chosen IV;
- chosen plaintext and chosen ciphertext;
- exact state reset between oracle calls.

Under that model, each IV induces a public permutation

$$F_{\mathrm{IV}} : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$$

that can be recovered in full with 2^{16} chosen encryptions. The inverse table can be recovered either by chosen-ciphertext queries or, as in the implementation used here, by inverting the public encryption table locally.

Main claim. For the tested random key

$$K = \texttt{2DE13EF0284A5C9BEB9FA9664ED8258BBDE5A7EA23C03CE6F3DD038E0E7570B5},$$

the public attack implemented in

`C:/Users/noaho/Desktop/upload/separ_public_full_hybrid_attack.c`

recovers the full 256-bit key by:

1. a chosen-IV beam search and exact hybrid public bootstrap for K_8;
2. on a successful inward IV, the same exact stage-local recursion for $(K_7, s_8), \ldots, (K_2, s_3)$;
3. an exact stage-1 closure from the public stage-2 reduced table;
4. exact full-codebook verification of the recovered candidate key.

What is proved here is therefore conditional exactness: once the chosen-IV search hits a suitable outer bootstrap IV and a suitable inward recursive IV, the rest of the attack is deterministic and exact.

2 Exact implementation model

2.1 Stage-local block permutation

Let the eight 32-bit stage keys be

$$K_i = (k_{i,0}, k_{i,1}) \in \mathbb{Z}_{2^{16}} \times \mathbb{Z}_{2^{16}}, \qquad 1 \le i \le 8.$$

Let $\Sigma : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$ be the nibblewise S-box layer induced by the fixed tables S_1, S_2, S_3, S_4 in the implementation, and let Σ^{-1} be the inverse layer induced by `IS1`, ..., `IS4`.

Let $L : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$ be the exact mixing layer implemented by `sep_rotl16`. If

$$x = (a, b, c, d) \in (\mathbb{Z}_{16})^4$$

in nibble notation, define

$$M(a, b, c, d) = (a \oplus c,\ b \oplus d,\ c \oplus b,\ d \oplus a),$$

identify nibble tuples with 16-bit words, and set

$$L(x) = M(x) \oplus \mathrm{rotl}_{16}(M(x), 12) \oplus \mathrm{rotl}_{16}(M(x), 8).$$

Its inverse is the exact `sep_inrotl16` routine.

For each stage i, the code derives two additional 16-bit subkeys $\kappa_{i,2}, \kappa_{i,3}$ from $(k_{i,0}, k_{i,1})$:

$$\begin{aligned} u_i &= \mathrm{rotl}_{16}(k_{i,0}, 6), & v_i &= \mathrm{rotl}_{16}(k_{i,1}, 10), \\ \kappa_{i,2} &= \big(u_i \vee (S_1((u_i \gg 6) \bmod 16) \ll 6)\big) \oplus (i+2), \\ \kappa_{i,3} &= \big(v_i \vee (S_1((v_i \gg 6) \bmod 16) \ll 6)\big) \oplus (i+3), \end{aligned}$$

where $\vee$ is bitwise OR and all arithmetic is in the exact 16-bit sense used by the implementation.

Define the XOR translation $A_t(x) = x \oplus t$. Then the stage-local encryption permutation implemented by `enc_block` is exactly

$$E_i = A_{\kappa_{i,2} \oplus \kappa_{i,3}} \circ \Sigma \circ A_{k_{i,0} \oplus k_{i,1}} \circ L \circ \Sigma \circ A_{\kappa_{i,3}} \circ L \circ \Sigma \circ A_{\kappa_{i,2}} \circ L \circ \Sigma \circ A_{k_{i,1}} \circ L \circ \Sigma \circ A_{k_{i,0}}, \quad (1)$$

and $D_i = E_i^{-1}$ is the exact `dec_block` inverse.

2.2 Translated stage action and full one-word encryption

For a state word $s_i \in \mathbb{Z}_{2^{16}}$, define the translated stage permutation

$$B_i^{(s_i)}(x) = E_i(x + s_i), \qquad \big(B_i^{(s_i)}\big)^{-1}(y) = D_i(y) - s_i,$$

where $+$ and $-$ are addition and subtraction in $\mathbb{Z}_{2^{16}}$.

For a fixed reset IV with state $(s_1, \ldots, s_8)$, the public one-word encryption permutation is

$$F_{\mathrm{IV}} = B_8^{(s_8)} \circ B_7^{(s_7)} \circ \cdots \circ B_1^{(s_1)}.$$

2.3 State update law

Write the eight state words as $(s_1, \ldots, s_8)$ and the stage outputs as

$$v_{12}, v_{23}, v_{34}, v_{45}, v_{56}, v_{67}, v_{78}, c.$$

From `Separ_Encryption` in `main.c`, the exact state update after one word is

$$s_2' = s_2 + v_{12} + v_{56} + s_6, \tag{2}$$
$$s_3' = s_3 + v_{23} + v_{34} + s_4 + s_1, \tag{3}$$
$$s_4' = s_4 + v_{12} + v_{45} + s_8, \tag{4}$$
$$s_5' = s_5 + v_{23} + L', \tag{5}$$
$$s_6' = s_6 + v_{12} + v_{45} + s_7, \tag{6}$$
$$s_7' = s_7 + v_{23} + v_{67}, \tag{7}$$
$$s_8' = s_8 + v_{45}, \tag{8}$$
$$s_1' = s_1 + s_5 + v_{34} + 2v_{23} + v_{78}. \tag{9}$$

The attack below uses only reset one-word tables, so these equations matter only because they determine which state word appears as the unknown output translation at each recursive step.

2.4 Oracle model

Under chosen IV and reset, each IV defines exact public codebooks

$$F_{\mathrm{IV}}(x) = \mathrm{Enc}_{K,\mathrm{IV}}(x), \qquad F_{\mathrm{IV}}^{-1}(y) = \mathrm{Dec}_{K,\mathrm{IV}}(y),$$

and the implementation used here obtains the inverse table locally by inverting the public encryption table.

3 Exact public tables used by the attack

3.1 Recursive current tables

For $m \in \{1, \ldots, 7\}$ define the inner cascade

$$R_m = B_m^{(s_m)} \circ B_{m-1}^{(s_{m-1})} \circ \cdots \circ B_1^{(s_1)}.$$

Definition 1 (Current table at level m)**.** *Assume $K_{m+1}, \ldots, K_8$ and $s_{m+2}, \ldots, s_8$ are already known for the fixed IV. The* current table *at level m is the public table*

$$C_m : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$$

computed recursively by

$$C_7 = D_8 \circ F_{\mathrm{IV}}, \qquad C_m = D_{m+1}\big(C_{m+1} - s_{m+2}\big) \quad (1 \le m \le 6).$$

Lemma 1 (Exact translation form of the current table)**.** *For every $m \in \{1, \ldots, 7\}$,*

$$C_m(x) = R_m(x) + s_{m+1} \qquad \textit{for all } x \in \mathbb{Z}_{2^{16}}.$$

Proof. For $m = 7$,

$$F_{\mathrm{IV}}(x) = E_8(R_7(x) + s_8),$$

hence

$$C_7(x) = D_8(F_{\mathrm{IV}}(x)) = R_7(x) + s_8.$$

Assume the statement holds at level $m+1$. Then

$$C_{m+1}(x) = R_{m+1}(x) + s_{m+2} = E_{m+1}(R_m(x) + s_{m+1}) + s_{m+2},$$

so

$$C_m(x) = D_{m+1}(C_{m+1}(x) - s_{m+2}) = D_{m+1}(E_{m+1}(R_m(x) + s_{m+1})) = R_m(x) + s_{m+1}.$$

□

Thus the exact public object attacked at level m is not an arbitrary hidden function. It is the inner cascade R_m observed through a single output translation by s_{m+1}.

3.2 Exact low-byte score

Let

$$H(y) = \left\lfloor \frac{y}{2^8} \right\rfloor \in \mathbb{Z}_{256}$$

be the output high byte. For a candidate low byte $b \in \mathbb{Z}_{256}$ and a row index $r \in \mathbb{Z}_{256}$ define

$$\sigma_{m,b}(r) = \left|\left\{H\big(C_m(256r + \lambda) - b\big) : \lambda \in \mathbb{Z}_{256}\right\}\right|,$$

and the exact total score

$$\Sigma_m(b) = \sum_{r=0}^{255} \sigma_{m,b}(r).$$

This is exactly the quantity computed by `exact_low_byte_scan_u16()`. The implementation evaluates all 256 low-byte hypotheses against all 256 rows and all 256 row entries.

3.3 Projected quotient cycles

Define the 4-bit projection

$$\pi(y) = H(y) \bmod 16 \in \mathbb{Z}_{16}.$$

For a stage key pair K_i define the stage-i quotient map

$$q_{i,K_i}(h) = \pi(E_i(256h)), \qquad h \in \mathbb{Z}_{16},$$

and the full high-byte quotient map

$$Q_{i,K_i}(h) = H(E_i(256h)), \qquad h \in \mathbb{Z}_{256}.$$

The implementation uses the fixed exact auxiliary sets

$$\Delta_0 = \{1, 2, 4, 8, 15, 16\},$$

$$\Omega = \{0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 200, 233, 255\},$$

and

$$\mathcal{R} = \{0x00, 0x55, 0xAA, 0xFF\}.$$

3.4 Representative tables and invisible bits

The code does not enumerate all 2^{32} stage pairs. It uses exact stage-specific invisible-bit sets. For stages $2, \ldots, 8$, the hard-coded sets are

$$\begin{aligned} P_2 = P_3 &= \{0, 1, 6, 7, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23\}, \\ P_4 &= \{6, 7, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23\}, \\ P_5 &= \{0, 6, 7, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23\}, \\ P_6 = P_7 &= \{0, 1, 2, 6, 7, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23\}, \\ P_8 &= \{0, 1, 6, 7, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23, 29\}. \end{aligned}$$

Let

$$V_i = \{0, \ldots, 31\} \setminus P_i.$$

Then

$$|V_2| = |V_3| = 18, \quad |V_4| = 20, \quad |V_5| = 19, \quad |V_6| = |V_7| = |V_8| = 17.$$

For each stage i, the representative table enumerates all visible masks in $\{0, 1\}^{V_i}$ with all invisible bits zero, computes the corresponding quotient cycle q_{i,K_i}, and groups visible assignments by that cycle. For stages $2, \ldots, 7$, the cycle is canonicalized by rotation so that the first entry is 0; stage 8 uses raw cycles.

3.5 Cycle edge score and exact order reconstruction

For any public table $T : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$, define the exact projected edge-count matrix

$$M_T(a, b) = \left|\{(x, \delta) \in \mathbb{Z}_{2^{16}} \times \Delta_0 : \pi(T(x)) = a,\ \pi(T(x + \delta)) = b\}\right|, \qquad a, b \in \mathbb{Z}_{16}.$$

If $c = (c_0, \ldots, c_{15})$ is a 16-cycle, define its exact edge score

$$S_T(c) = \sum_{j=0}^{15} M_T(c_j, c_{j+1 \bmod 16}).$$

The cycle score recovers only the 16-point projected cycle. To lift it to a full 256-point high-byte ordering, the implementation builds the exact 8-bit transition matrix

$$N_T(u, v) = \left|\{(x, \delta) \in \mathbb{Z}_{2^{16}} \times \Delta_0 : H(T(x)) = u,\ H(T(x + \delta)) = v\}\right|, \qquad u, v \in \mathbb{Z}_{256},$$

and then, for each adjacent pair of cycle layers, solves an exact 16×16 maximum-weight assignment by the Hungarian algorithm. The resulting wrap permutation is accepted only if it forms one 16-cycle on the path labels. This yields the exact order array used by the candidate lift.

3.6 Exact candidate lift and verifier

For any recovered cycle/order pair, the implementation:

1. enumerates every representative-group base key compatible with the cycle;
2. expands all invisible masks in P_i;
3. computes the full quotient bytes Q_{i,K_i};
4. tests exact equality against the reconstructed order targets first on the necessary sample set Ω, then on all 256 values;

5. for stages $2, \ldots, 7$, sets
$$s_{i+1} = \mathrm{lo}(s_{i+1}) + 256\tau$$
from the recovered byte translation τ;

6. exact-scores the surviving candidates by
$$V_i(K_i; T) = \sum_{r \in \mathcal{R}} \min_{b \in \mathbb{Z}_{256}} \left| \left\{ H\big(D_i(T(256r + \lambda)) - b\big) : \lambda \in \mathbb{Z}_{256} \right\} \right| .$$

The sample test is not heuristic: full equality on $\mathbb{Z}_{256}$ implies equality on Ω, so the sample check discards only impossible candidates.

4 The outer public bootstrap for K_8

4.1 Weak-IV beam search

For an IV, define the exact row score
$$\rho(\mathrm{IV}) = \left| \{ H(F_{\mathrm{IV}}(\lambda)) : \lambda \in \mathbb{Z}_{256} \} \right| .$$

The implementation keeps the B IVs with smallest $\rho(\mathrm{IV})$. This is a search-ordering device only. It is not part of the acceptance criterion.

4.2 Hybrid exact bootstrap

For each IV in the beam, the implementation:

1. computes the exact stage-8 quotient/bootstrap candidate set by projected-cycle maximization and exact verifier scoring;

2. closes that set under the independent MSB toggles
$$(a, b) \mapsto (a \oplus \epsilon_0 2^{15},\, b \oplus \epsilon_1 2^{15});$$

3. exact-scores the resulting small pool by the additive-differential score
$$A_{\mathrm{IV}}(K_8) = \sum_{\delta \in \Delta_0} \max_{\Delta \in \mathbb{Z}_{2^{16}}} \left| \{ x : D_8(F_{\mathrm{IV}}(x + \delta); K_8) - D_8(F_{\mathrm{IV}}(x); K_8) = \Delta \} \right| ;$$

4. aggregates those exact scores across the beam and outputs the best K_8.

   For the tested random key, the 512-trial beam with beam size 16 recovered the correct
$$K_8 = (\texttt{0E75}, \texttt{70B5})$$

on search seeds $1, 2, 3$. This is empirical evidence, not a proof of key-generic stage-8 success.

5 Exact same-stage recursion from K_7 to K_2

5.1 Deterministic recursion once K_8 is known

Fix an IV and assume the outer bootstrap has already recovered the correct K_8. The inward recursion then repeatedly applies the exact stage-local scan to the current tables of Lemma 1.

For $m = 7, 6, 5, 4, 3, 2$:

1. compute the current table C_m;
2. compute the exact low-byte score Σ_m and let ℓ_m be its minimizing low byte;
3. form the corrected table
$$\widetilde{C}_m(x) = C_m(x) - \ell_m;$$
4. compute all maximizing projected cycles for $\widetilde{C}_m$;
5. reconstruct all exact compatible orders;
6. lift to the full candidate family (K_m, s_{m+1});
7. exact-score all candidates by the verifier V_m and output the lexicographic winner;
8. peel the recovered stage:
$$C_{m-1} = D_m(C_m - s_{m+1}; K_m).$$

Definition 2 (Recursive IV)**.** *An IV is* recursive to depth d *if, when the above deterministic procedure is run with the correct K_8, it returns the true pairs/states for stages*

$$7, 6, \ldots, d.$$

An IV is fully inward-recursive *if it is recursive to depth* 2.

Why the recursion stops at stage 2. The projected-family scan exists only for stages $2, \ldots, 8$. Stage 1 has no next visible stage and therefore no stage-1 analogue of the quotient cycle. Stage 1 must be closed separately from the stage-2 reduced table.

6 Exact stage-1 closure from the public stage-2 table

6.1 Reduction to a single translated stage

Assume the inward recursion has recovered the true (K_2, s_3) on a fully inward-recursive IV. Then
$$C_2(x) = E_2(E_1(x + s_1) + s_2) + s_3.$$
Define
$$W_2(x) = D_2(C_2(x) - s_3; K_2).$$
Then exactly
$$W_2(x) = E_1(x + s_1) + s_2. \tag{10}$$

6.2 Low-byte recovery for s_2

The implementation first applies the same exact low-byte scan to W_2. For each candidate
$$s_2^\star \in \mathbb{Z}_{2^{16}},$$
it forms
$$C^\star(x) = W_2(x) - s_2^\star.$$
Any true stage-1 solution must satisfy
$$C^\star(x) = E_1(x + s_1; K_1).$$

6.3 Step 1: exact row-signature inversion

For $u \in \mathbb{Z}_{256}$ define the 16-point row signature

$$R_u^{C^\star}(h) = \pi(C^\star(u + 256h)), \qquad h \in \mathbb{Z}_{16}.$$

Write

$$s_1 = s_{1,\mathrm{lo}} + 256\, s_{1,\mathrm{mid}} + 4096\, s_{1,\mathrm{hi}},$$

with $s_{1,\mathrm{lo}} \in \mathbb{Z}_{256}$ and $s_{1,\mathrm{mid}}, s_{1,\mathrm{hi}} \in \mathbb{Z}_{16}$.

For a candidate $s_{1,\mathrm{lo}}$, the routine `stage1_step1_candidates_u16()` checks the exact carry partition:

> all rows with $u + s_{1,\mathrm{lo}} < 256$ must share one common signature σ, and all rows with $u + s_{1,\mathrm{lo}} \geq 256$ must share the one-step rotation $\mathrm{rot}_1(\sigma)$.

For each surviving $s_{1,\mathrm{lo}}$ and each $s_{1,\mathrm{mid}} \in \mathbb{Z}_{16}$, the implementation rotates σ by $16 - s_{1,\mathrm{mid}}$ and inverts it through the exact precomputed family

$$\nu_{a,b,c,d} : \mathbb{Z}_{16} \to \mathbb{Z}_{16},$$

where

$$\nu_{a,b,c,d}(x) = S_2\big(t_4(x) \oplus a \oplus b\big) \oplus c \oplus d$$

with

$$\begin{aligned} t_1(x) &= S_2(x \oplus a), \\ t_2(x) &= S_2(t_1(x) \oplus b), \\ t_3(x) &= S_2(t_2(x) \oplus c), \\ t_4(x) &= S_2(t_3(x) \oplus d). \end{aligned}$$

This yields exact candidates for

$$s_{1,\mathrm{lo}}, \quad s_{1,\mathrm{mid}}, \quad a, \quad b, \quad c, \quad d.$$

6.4 Step 2: exact lift to full K_1 candidates

The routine next uses exact inverse-image tables for the derived-key high nibbles of $\kappa_{1,2}$ and $\kappa_{1,3}$. For each surviving $(s_{1,\mathrm{lo}}, s_{1,\mathrm{mid}}, a, b, c, d)$, each

$$\ell_6 \in I_2(c), \qquad \beta \in I_3(d),$$

and each 12-bit free mask $f \in \mathbb{Z}_{2^{12}}$, it constructs

$$k_{1,0} = \ell_6 \;\vee\; \big((f \bmod 4) \ll 6\big) \;\vee\; (a \ll 8) \;\vee\; \left(\left(\left\lfloor \frac{f}{4} \right\rfloor \bmod 16\right) \ll 12\right), \tag{11}$$

$$k_{1,1} = (\beta \bmod 4) \;\vee\; \left(\left(\left\lfloor \frac{f}{64} \right\rfloor \bmod 64\right) \ll 2\right) \;\vee\; (b \ll 8) \;\vee\; \left(\left(\left\lfloor \frac{\beta}{4} \right\rfloor \bmod 16\right) \ll 12\right). \tag{12}$$

This is the exact bit lift implemented by `stage1_step2_candidates_u16()`.

The resulting pair is exact-tested on the 12-bit slice

$$X_{12} = \{0, \ldots, 4095\}$$

by comparing nibble (7..4) values:

$$\left\lfloor \frac{E_1((x + s_{1,\mathrm{lo}} + 256 s_{1,\mathrm{mid}}) \bmod 2^{12}; K_1)}{2^4} \right\rfloor \bmod 16 = \left\lfloor \frac{C^\star(x)}{2^4} \right\rfloor \bmod 16 \quad \text{for all } x \in X_{12}.$$

6.5 Step 3: exact recovery of the top nibble of s_1

For each surviving pair K_1, the implementation scans all 16 possibilities for $s_{1,\mathrm{hi}}$ and performs full exact verification:

$$E_1(x + s_1; K_1) = C^\star(x) \qquad \text{for all } x \in \mathbb{Z}_{2^{16}}.$$

If exactly one triple (K_1, s_1, s_2) survives, the stage-1 closure succeeds.

Theorem 1 (Exactness of the stage-1 closure)**.** *Let W_2 be the exact public table of* (10)*. If there is a unique triple (K_1, s_1, s_2) satisfying*

$$W_2(x) = E_1(x + s_1; K_1) + s_2 \qquad \textit{for all } x \in \mathbb{Z}_{2^{16}},$$

then `stage1_finish_from_stage2_current()` *returns exactly that triple.*

Proof. The routine is exhaustive over all candidates for the low and high bytes of s_2, all exact row-signature solutions for $(s_{1,\mathrm{lo}}, s_{1,\mathrm{mid}}, a, b, c, d)$, all inverse-image candidates from $I_2(c)$ and $I_3(d)$, all 2^{12} free masks in (11)–(12), and all 16 possibilities for $s_{1,\mathrm{hi}}$. Every surviving candidate is tested by exact equality either on the 12-bit slice or on the full 2^{16}-entry table, and the routine accepts only if the number of full solutions is exactly one. Hence a unique solution is returned exactly. □

7 Public full-key acceptance and conditional correctness

Definition 3 (Exact public acceptance test)**.** *Given recovered pairs*

$$\widehat{K} = (\widehat{K}_1, \ldots, \widehat{K}_8),$$

rebuild the candidate one-word table

$$\widehat{F}_{\mathrm{IV}}$$

under the same IV and accept if and only if

$$\widehat{F}_{\mathrm{IV}} = F_{\mathrm{IV}}$$

as functions on $\mathbb{Z}_{2^{16}}$.

Because the block size is only 16 bits, this is an exact comparison of two 65536-entry tables.

Definition 4 (Outer-success and inward-success events)**.** *Fix the exact IV sequence tested by the implementation.*

(a) The outer-success event *is the event that the beam search and exact hybrid aggregate rank the true K_8 first.*

(b) The inward-success event *for an IV is the event that the same-stage recursion returns the true $(K_7, s_8), \ldots, (K_2, s_3)$ and the stage-1 solver returns a unique solution.*

Theorem 2 (Conditional exactness of the full public attack)**.** *Assume the outer-success event occurs on one of the tested beam IVs, and assume one of the tested inward IVs satisfies the inward-success event. Then the implementation outputs the true 256-bit key and accepts it by the exact codebook test. No false full key can be accepted on that IV.*

Proof. Under outer-success, the recovered K_8 is the true K_8. By Lemma 1, each inward recursion level is then applied to the exact public current table $C_m = R_m + s_{m+1}$. Under inward-success, the recursion returns the true pairs/states down to (K_2, s_3). Equation (10) therefore holds for the true W_2, and Theorem 1 returns the unique true stage-1 triple (K_1, s_1, s_2). The reconstructed 256-bit key is thus the true implementation key. The final acceptance test compares full tables, so no false key can satisfy it on the successful IV. □

8 Worked random-key recovery

8.1 Target key and successful IVs

The tested random key is

$$K = \texttt{2DE13EF0284A5C9BEB9FA9664ED8258BBDE5A7EA23C03CE6F3DD038E0E7570B5}.$$

The successful public run used:

- outer beam budget $N_{\text{out}} = 512$;
- beam size $B = 16$;
- search seed 1;
- successful inward IV
$$\texttt{D17A5783A9F94E08F948B6EB536519A4}.$$

8.2 Recovered values

The public recovery returned

$$\begin{aligned}
K_8 &= (\texttt{0E75}, \texttt{70B5}), \\
K_7 &= (\texttt{F3DD}, \texttt{038E}), & s_8 &= \texttt{BE8C}, \\
K_6 &= (\texttt{23C0}, \texttt{3CE6}), & s_7 &= \texttt{E981}, \\
K_5 &= (\texttt{BDE5}, \texttt{A7EA}), & s_6 &= \texttt{EA65}, \\
K_4 &= (\texttt{4ED8}, \texttt{258B}), & s_5 &= \texttt{44A8}, \\
K_3 &= (\texttt{EB9F}, \texttt{A966}), & s_4 &= \texttt{E26E}, \\
K_2 &= (\texttt{284A}, \texttt{5C9B}), & s_3 &= \texttt{BA5B}, \\
K_1 &= (\texttt{2DE1}, \texttt{3EF0}), & s_2 &= \texttt{461F}, & s_1 &= \texttt{DF21}.
\end{aligned}$$

The decisive public log line is

```
full-key candidate verified against exact one-word oracle table
```

from `separ_public_full_hybrid_attack_clean_verbose.log`.

9 Monte Carlo estimate for IV-search success

9.1 Deterministic core versus stochastic front-end

The attack splits into:

- a deterministic algebraic core: once a suitable IV is fixed, all recovery and verification steps are exact;
- a stochastic IV-search front-end: the attack samples IVs until outer-success and inward-success occur.

For a fixed recovered K_8, model the inward search as a Bernoulli process with success probability

$$p_{\text{in}} = \Pr[\text{a random IV is fully inward-recursive} \mid K_8 \text{ already recovered}].$$

9.2 Pilot estimate

Pooling the earlier inward-recursion pilot scans with the first observed full success gives

$$x = 1 \text{ success in } n = 1727 \text{ random inward IV trials,}$$

hence

$$\widehat{p}_{\text{in}} = \frac{1}{1727} \approx 5.79 \times 10^{-4}.$$

The Wilson 95% interval is approximately

$$0.000102 \lesssim p_{\text{in}} \lesssim 0.003273.$$

These values are empirical measurements for the tested random-key instance, not a theorem about arbitrary keys.

9.3 Budget-to-success conversion

For an inward IV budget of N independent random trials,

$$P_{\text{succ}}(N) = 1 - (1 - \widehat{p}_{\text{in}})^N.$$

The corresponding point estimates are:

Inward IV budget N	Point estimate	Wilson low	Wilson high
256	0.138	0.026	0.568
512	0.257	0.051	0.813
1024	0.447	0.099	0.965
2048	0.695	0.189	0.999
4096	0.907	0.342	1.000
8192	0.991	0.567	1.000
10000	0.997	0.640	1.000

10 Complexity and bound

10.1 Oracle-query complexity

One full one-word codebook requires exactly

$$2^{16}$$

chosen encryptions. The implementation does not require chosen-ciphertext queries, so

$$Q_{\text{dec}} = 0.$$

Suppose the attack is run with:

- outer IV-search budget N_{out};
- beam size B;
- one distinguished initial inward IV;
- inward random-IV budget N_{in}.

Then the worst-case number of codebooks built by the current implementation is

$$N_{\text{out}} + B + 1 + B + N_{\text{in}} = N_{\text{out}} + 2B + N_{\text{in}} + 1.$$

Hence the exact chosen-encryption query bound is

$$Q_{\text{enc}} = \big(N_{\text{out}} + 2B + N_{\text{in}} + 1\big)2^{16}.$$

For the validated successful run reported above,

$$N_{\text{out}} = 512, \qquad B = 16, \qquad N_{\text{in}} = 1,$$

and success occurred on the distinguished initial inward IV. The realized query count was therefore

$$(512 + 16 + 1) \cdot 2^{16} = 529 \cdot 65536 = 34{,}668{,}544.$$

10.2 Exact local work

Once a codebook is built, all remaining work is local and exact. The fixed representative-table sizes used by the implementation are:

Stage	$\lvert V_i \rvert$	Representative masks
8	17	$2^{17} = 131072$
7	17	$2^{17} = 131072$
6	17	$2^{17} = 131072$
5	19	$2^{19} = 524288$
4	20	$2^{20} = 1048576$
3	18	$2^{18} = 262144$
2	18	$2^{18} = 262144$

On the successful inward IV, the cycle-search logs recorded:

Stage	Representative/cycle space	Logged cycle work
7	2^{17} reps, shifted cycle search	$2^{19} = 524288$
6	2^{17} reps, shifted cycle search	$2^{19} = 524288$
5	2^{19} reps, shifted cycle search	$2^{20} = 1048576$
4	2^{20} reps, shifted cycle search	$2^{20} = 1048576$
3	2^{18} reps, shifted cycle search	$3 \cdot 2^{18} = 786432$
2	2^{18} reps, shifted cycle search	$3 \cdot 2^{18} = 786432$

The stage-1 closure then adds:

1. an exact low-byte scan for s_2;
2. all 256 high-byte hypotheses for s_2;
3. exact inversion of the row-signature predicate;
4. exact lift over the remaining 2^{12} free key bits;
5. full table verification of the surviving stage-1 candidates.

10.3 Overall bound

For fixed budgets $(N_{\text{out}}, B, N_{\text{in}})$, the implemented attack has exact query complexity

$$Q_{\text{enc}} = \big(N_{\text{out}} + 2B + N_{\text{in}} + 1\big)2^{16}, \qquad Q_{\text{dec}} = 0,$$

and deterministic local work equal to one execution of the fixed representative-table builds for stages $2, \ldots, 8$ plus, for each tested IV, a constant number of exact 2^{16}-table passes together with the stage-dependent cycle, rotation, verifier, and stage-1 scans listed above.

As an empirical orientation point rather than an asymptotic claim, on the user's machine one measured full run with

$$N_{\text{out}} = 10000, \qquad B = 16, \qquad N_{\text{in}} = 1$$

and the successful distinguished inward IV supplied on the command line completed in approximately 125.21 seconds of wall-clock time.

11 Implementation and runtime

The attack is implemented in the multithreaded C program

`separ_public_full_hybrid_attack.exe`.

The public end-to-end runner is

```
separ_public_full_hybrid_attack.exe --search-trials N --search-beam B
    --search-seed S --inward-trials M --iv HEX32 --oracle-key HEX64
```

with optional `--verbose` output. In the default mode the program shows only the current phase, progress, best IV, current IV, and best recovered key/state values in a compact in-place display.

Operationally, the cleaned public path:

- recovers K_8 only by the public hybrid score over the beam;
- recovers stages 7 through 2 only by the exact public recursion above;
- recovers stage 1 only by the exact closure of Theorem 1;
- accepts a full key only after exact equality of the recovered codebook with the oracle codebook.

12 Conclusion

The `main.c` implementation of SEPAR is broken end-to-end in the chosen-IV, chosen-plaintext, chosen-ciphertext, reset model. The critical structural fact is that, once a suitable IV is found, the implementation exposes a chain of exact public reduced tables in which each stage appears as a translated 16-bit permutation, and those translations can be stripped or solved exactly.

The resulting attack is not a heuristic key-ranking experiment. It is a deterministic full-key recovery pipeline conditioned on the success of the IV-search phase. The only remaining uncertainty is empirical: the probability that a random chosen IV falls into the recursive classes required by the outer bootstrap and the inward recursion.